Cheatsheet — AJAX y Arquitecturas Cliente-Servidor

> **Materia:** Programación Orientada a la Web
> **Uso:** repaso rápido y consulta durante el parcial.
> **Base:** teoría de la cátedra (XMLHttpRequest clásico) + Fetch API moderna.

Índice

1. Qué es AJAX
2. Tecnologías que lo componen
3. Modelo tradicional vs. modelo AJAX
4. Cómo funciona: el ciclo completo
5. El objeto `XMLHttpRequest`
6. Métodos de `XMLHttpRequest`
7. Propiedades de `XMLHttpRequest`
8. `readyState` y `status`
9. `onreadystatechange` vs. `onload`
10. GET vs. POST
11. Asíncrono (`true`) vs. síncrono (`false`)
12. Primer ejemplo completo
13. Ejemplo: modificar un `div`
14. Ejemplo: menú que carga contenidos
15. AJAX con formularios
16. Ejemplo: validar un nickname
17. AJAX con librerías (Prototype)
18. Fetch API (método moderno)
19. Same Origin Policy y CORS
20. Códigos de estado HTTP
21. Ventajas, desventajas y consideraciones
22. Errores típicos y checklist
23. Plantilla completa

1. Qué es AJAX

AJAX = *Asynchronous JavaScript + XML* ("JavaScript asíncrono + XML").

> "AJAX no es una tecnología en sí mismo (no es un lenguaje de programación). En realidad, se trata de varias tecnologías independientes que se unen de formas nuevas y sorprendentes."

- Es una **técnica de desarrollo web** para crear aplicaciones interactivas o **RIA** (*Rich Internet Applications*).
- La aplicación se ejecuta en el **cliente** (navegador) y se comunica de forma **asíncrona** con el **servidor** en segundo plano.
- Permite **modificar partes de la página sin recargarla**, mejorando interactividad, velocidad y usabilidad.

Concepto clave	Significado
Asíncrono	Los datos se piden y cargan en segundo plano sin bloquear la página.
Sin recarga	El usuario no ve una ventana vacía esperando respuesta.
Basado en estándares	Funciona en múltiples plataformas, sistemas operativos y navegadores.

2. Tecnologías que lo componen

Tecnología	Rol dentro de AJAX
XHTML / HTML	Estructura y contenido.
CSS	Presentación basada en estándares.
DOM	Interacción y manipulación dinámica de la presentación.
XML / XSLT / JSON	Intercambio y manipulación de información (también texto plano).

Tecnología	Rol dentro de AJAX
`XMLHttpRequest`	Intercambio asíncrono de información con el servidor.
JavaScript	Une todas las demás tecnologías (y habla con PHP, ASP, etc.).

```
      XHTML   CSS
         \     /
          DOM
           |
    XML / JSON / texto
           |
     XMLHttpRequest
           |
       JAVASCRIPT
```

3. Modelo tradicional vs. modelo AJAX

Modelo tradicional

1. El usuario realiza una acción (clic en un botón, elegir una opción).
2. El navegador envía la petición al servidor.
3. El servidor procesa y devuelve una **página HTML completa**.
4. El navegador **recarga** toda la página.

Consecuencia: el usuario espera, la pantalla se pone en blanco y se pierde el estado de la página.

Modelo AJAX

1. Ocurre un evento en el navegador.
2. Se crea el objeto `XMLHttpRequest`.
3. Se envía la petición HTTP al servidor.
4. El servidor la procesa y responde **en segundo plano**.
5. JavaScript procesa la respuesta.
6. Se actualiza **solo la parte necesaria** del DOM.

Consecuencia: no hay recarga, la interacción no se interrumpe y la respuesta se percibe inmediata.

Aspecto	Tradicional	AJAX
Recarga de página	Sí, completa	No
Espera del usuario	Visible (pantalla en blanco)	No visible
Datos transferidos	Página HTML completa	Solo los datos necesarios
Estado de la página	Se pierde	Se conserva
Interactividad	Baja	Alta
Uso del servidor	Renderiza HTML	Puede delegar el render al cliente

> Las peticiones HTTP completas se reemplazan por **peticiones JavaScript** al elemento encargado de AJAX. Las peticiones más simples ni siquiera requieren intervención del servidor, por lo que la respuesta es inmediata.

4. Cómo funciona: el ciclo completo

```
NAVEGADOR (cliente)                          SERVIDOR
   |                                            |
   | 1. Ocurre un evento                        |
   | 2. Se crea XMLHttpRequest                  |
   | 3. open() + send()  --- HTTP Request --->  |
   |                                            | 4. Procesa la petición
   | <--- HTTP Response (texto/XML/JSON) -----  | 5. Crea la respuesta
   | 6. JavaScript procesa la respuesta         |
   | 7. Actualiza el contenido del DOM          |
```

Los **cuatro bloques** de toda aplicación AJAX:

1. **Instanciar** el objeto `XMLHttpRequest`.
2. **Preparar** la función de respuesta.
3. **Realizar** la petición al servidor.

4. **Ejecutar** la función de respuesta cuando llega la respuesta.

5. El objeto `XMLHttpRequest`

Es el objeto clave: permite comunicarse con el servidor en segundo plano sin recargar la página.

La implementación depende de cada navegador, por eso la forma clásica de crearlo incluye una verificación:

```
var peticion_http;

if (window.XMLHttpRequest) {
  // Navegadores que siguen los estándares
  peticion_http = new XMLHttpRequest();
} else if (window.ActiveXObject) {
  // Navegadores obsoletos (Internet Explorer antiguo)
  peticion_http = new ActiveXObject("Microsoft.XMLHTTP");
}
```

Versión con `try/catch` para las distintas versiones de IE:

```
function nuevoAjax() {
  var xmlhttp = false;
  if (window.XMLHttpRequest) {
    xmlhttp = new XMLHttpRequest();          // Opera, Safari, Firefox, Chrome
  } else if (window.ActiveXObject) {
    try {
      xmlhttp = new ActiveXObject("Msxml2.XMLHTTP");
    } catch (e) {
      try {
        xmlhttp = new ActiveXObject("Microsoft.XMLHTTP");
      } catch (e) {
        xmlhttp = false;
      }
    }
  }
  return xmlhttp;
}
```

> Hoy alcanza con `const xhttp = new XMLHttpRequest();` . El código de compatibilidad se mantiene porque la cátedra lo muestra.

6. Métodos de `XMLHttpRequest`

`open(método, URL, asíncrono)`

Prepara la petición. **No la envía.**

```
peticion_http.open('GET', 'http://localhost/prueba.txt', true);
```

Parámetro	Descripción
`método`	`'GET'` o `'POST'` (también `PUT`, `DELETE`, etc.).
`URL`	Dirección del recurso solicitado.
`asíncrono`	`true` = asíncrona (recomendado), `false` = síncrona.

`send(datos)`

Envía la petición.

```
peticion_http.send(null);        // GET: sin cuerpo
peticion_http.send("t1=" + t1);  // POST: datos en el cuerpo
```

`setRequestHeader(nombre, valor)`

Agrega cabeceras HTTP. **Obligatorio** para POST con formulario:

```
peticion_http.setRequestHeader("Content-Type",
                               "application/x-www-form-urlencoded");
```

`abort()`

Cancela la petición en curso.

`getAllResponseHeaders()` / `getResponseHeader(nombre)`

Devuelven las cabeceras de la respuesta.

7. Propiedades de `XMLHttpRequest`

Propiedad	Descripción
`readyState`	Valor numérico entero con el estado de la petición (0 a 4).
`responseText`	Contenido de la respuesta como **cadena de texto**.
`responseXML`	Contenido de la respuesta en formato **XML** (se procesa como DOM).
`status`	**Código de estado HTTP** devuelto por el servidor (200, 404, 500...).
`statusText`	El mismo estado, pero como texto: `"OK"`, `"Not Found"`, etc.
`onreadystatechange`	Función que se ejecuta **cada vez** que cambia `readyState`.
`onload`	Función que se ejecuta **solo** cuando la respuesta terminó de cargar.
`onerror`	Función que se ejecuta si la petición falla.
`timeout` / `ontimeout`	Tiempo máximo de espera y su manejador.

8. `readyState` y `status`

Valores de `readyState`

Valor	Nombre	Descripción
`0`	No inicializado	Objeto creado, pero no se invocó `open()`.
`1`	Cargando	Se invocó `open()`, pero no `send()`.
`2`	Cargado	Se invocó `send()`, pero el servidor aún no respondió.
`3`	Interactivo	Se recibieron algunos datos; todavía no se puede usar `responseText`.
`4`	Completo	Se recibieron **todos** los datos de la respuesta.

> La condición habitual es `readyState == 4`, porque en ese momento la respuesta está disponible.

Códigos `status` más comunes

Código	Significado
`200`	OK — respuesta correcta.
`201`	Created — recurso creado.
`301` / `302`	Redirección permanente / temporal.
`304`	Not Modified — sin cambios (caché).
`400`	Bad Request — petición mal formada.
`401`	Unauthorized — no autenticado.
`403`	Forbidden — sin permisos.
`404`	Not Found — recurso no encontrado.
`500`	Internal Server Error — error del servidor.

```
if (peticion_http.readyState == 4 && peticion_http.status == 200) {
  // respuesta correcta: procesar
}
```

9. `onreadystatechange` vs. `onload`

`onreadystatechange`

Se ejecuta en **cada cambio** de estado y permite conocer el estado de la llamada.

```
peticion_http.onreadystatechange = muestraContenido;
```

```
function muestraContenido() {
  if (peticion_http.readyState == 4) {
    if (peticion_http.status == 200) {
      alert(peticion_http.responseText);
    }
  }
}
```

También admite una función anónima:

```
xhttp.onreadystatechange = function () {
  if (this.readyState == 4 && this.status == 200) {
    document.getElementById("demo").innerHTML = this.responseText;
  }
};
```

`onload`

Se ejecuta **una sola vez**, cuando la respuesta ya está cargada. No contempla los estados intermedios: es más corto, pero no permite reaccionar a los estados 1, 2 y 3.

```
peticion_http.onload = muestraContenido;

function muestraContenido() {
  alert(peticion_http.responseText);
}
```

	`onreadystatechange`	`onload`
Se ejecuta	En cada cambio de estado	Solo al terminar la carga
Permite ver estados intermedios	Sí	No
Verifica `status`	Se hace a mano	También conviene hacerlo
Código	Más largo	Más corto y moderno

10. GET vs. POST

GET simple (sin parámetros)

```
peticion_http.open('GET', 'http://localhost/prueba.txt', true);
peticion_http.send(null);
```

GET con parámetros

Los datos viajan **en la URL**:

```
ajax.open("GET", "ejemploajax2.php?variable=valor", true);
// varios parámetros: ?a=1&b=2
```

POST con parámetros

Los datos viajan en el **cuerpo** de la petición:

```
ajax.open("POST", "ejemploajax2.php", true);
ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
ajax.send("t1=" + t1 + "&t2=" + t2);
```

Criterio	GET	POST
Velocidad / simplicidad	Más simple y rápido	Un poco más pesado
Dónde van los datos	En la URL	En el cuerpo
Límite de tamaño	Limitado por la URL	Sin limitaciones prácticas
Visibilidad	Visible en la URL y el historial	No visible
Caché	Puede usar caché	Normalmente no
Seguridad de datos sensibles	Mala	Mejor
Uso recomendado	Consultar / recuperar datos	Enviar datos, actualizar archivos o BD

> **Regla de la cátedra:** usar POST siempre que se actualice un archivo o una base de datos, se envíen muchos datos o se envíe entrada del usuario (más robusto y seguro que GET).

11. Asíncrono (`true`) vs. síncrono (`false`)

```
peticion_http.open('GET', 'datos.txt', true);   // asíncrona
peticion_http.open('GET', 'datos.txt', false);  // síncrona
```

	Asíncrona (`true`)	Síncrona (`false`)
JavaScript espera	No	Sí, se bloquea
Interacción del usuario	Continúa	Se congela
Rendimiento	Bueno	Malo
Recomendación	**Usar siempre**	Evitar

> Con `false` , si el servidor es lento, la ejecución de JavaScript se detiene hasta que llegue la respuesta. La función puede colgarse o dejar de ejecutarse.

12. Primer ejemplo completo

Adaptación del clásico "Hola Mundo": descargar un archivo del servidor y mostrar su contenido sin recargar.

```
var peticion_http;

// 1. Crear el objeto
if (window.XMLHttpRequest) {
  peticion_http = new XMLHttpRequest();
} else if (window.ActiveXObject) {
  peticion_http = new ActiveXObject("Microsoft.XMLHTTP");
}

// 2. Preparar la función de respuesta
peticion_http.onreadystatechange = muestraContenido;

function muestraContenido() {
  if (peticion_http.readyState == 4 && peticion_http.status == 200) {
    alert(peticion_http.responseText);
  }
}

// 3. Realizar la petición
peticion_http.open('GET', 'http://localhost/prueba.txt', true);
peticion_http.send(null);
```

13. Ejemplo: modificar un `div`

```
<div id="demo">
  <h1>AJAX cambiando el contenido de una página</h1>
  <button type="button" onclick="cargar_documento()">
    Cargar archivo holamundo.txt
  </button>
</div>

<script>
function cargar_documento() {
  var xhttp = new XMLHttpRequest();

  xhttp.onreadystatechange = function () {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML = this.responseText;
    }
  };

  xhttp.open("GET", "holamundo.txt", true);
  xhttp.send();
}
</script>
```

Variante con `onload`

```
function loadDoc() {
  const xhttp = new XMLHttpRequest();
```

```
  xhttp.onload = function () {
    document.getElementById("demo").innerHTML = this.responseText;
  };

  xhttp.open("GET", "holamundo.txt");
  xhttp.send();
}
```

14. Ejemplo: menú que carga contenidos

Se cargan los contenidos de los enlaces sin volver a cargar el menú.

```
<div id="menu">
  <a href="javascript:llamadasin('ejemplos/ej1.html', 'contenido');">Ejemplo 1</a>
  <a href="javascript:llamadasin('ejemplos/ej2.html', 'contenido');">Ejemplo 2</a>
</div>

<div id="contenido">
  <h3 class="titulo">Aquí se cargará asíncronamente el contenido</h3>
  <p>Elegí una opción del menú...</p>
</div>
```

```
function llamadasin(url, contenedor) {
  var pagina_requerida = false;

  if (window.XMLHttpRequest) {
    pagina_requerida = new XMLHttpRequest();
  } else if (window.ActiveXObject) {
    try {
      pagina_requerida = new ActiveXObject("Msxml2.XMLHTTP");
    } catch (e) {
      try {
        pagina_requerida = new ActiveXObject("Microsoft.XMLHTTP");
      } catch (e) {
        pagina_requerida = false;
      }
    }
  }

  pagina_requerida.onreadystatechange = function () {
    pintapagina(pagina_requerida, contenedor);
  };

  pagina_requerida.open('GET', url, true);
  pagina_requerida.send(null);
}

function pintapagina(pagina_requerida, contenedor) {
  if (pagina_requerida.readyState == 4 &&
      (pagina_requerida.status == 200 ||
       window.location.href.indexOf("http") == -1)) {
    document.getElementById(contenedor).innerHTML =
      pagina_requerida.responseText;
  }
}
```

> El chequeo extra `window.location.href.indexOf("http") == -1` permite que funcione también abierto como archivo local (`file://`), donde no hay código de estado HTTP.

15. AJAX con formularios

Hay que tomar **cada campo** del formulario y concatenarlo como `variable=valor`.

```
function cargarContenido() {
  var t1, t2, contenedor;

  contenedor = document.getElementById('contenedor');
  t1 = document.getElementById('texto1').value;
  t2 = document.getElementById('texto2').value;

  ajax = nuevoAjax();
```

```
  ajax.open("POST", "ejemploajax2.php", true);

  ajax.onreadystatechange = function () {
    if (ajax.readyState == 4) {
      contenedor.innerHTML = ajax.responseText;
    }
  };

  ajax.setRequestHeader("Content-Type",
                        "application/x-www-form-urlencoded");
  ajax.send("t1=" + t1 + "&t2=" + t2);
}
```

Equivalente con GET

```
ajax.open("GET", "ejemploajax2.php?variable=valor", true);
ajax.send(null);
```

Serializar todos los campos

```
var formulario = document.getElementById("formulario");
var datos = new FormData(formulario);

// Con XMLHttpRequest: NO se setea Content-Type a mano
ajax.open("POST", "procesar.php", true);
ajax.send(datos);
```

Forma	Cuándo usarla
Concatenar `clave=valor&...`	Pocos campos de texto. Hay que escapar valores.
`FormData`	Formularios completos o con archivos.
Librería (jQuery, Prototype)	Cuando ya se usa la librería.

---

16. Ejemplo: validar un nickname

Se consulta la base de datos **mientras el usuario escribe**, sin enviar el formulario.

```
<form>
  Nickname:
  <input type="text" onkeyup="comprobar_nick(this.value)">
  <span id="mensaje"></span>
</form>
```

```
function comprobar_nick(str) {
  if (str.length == 0) {
    document.getElementById("mensaje").innerHTML = "";
    return;
  }

  var xmlhttp = new XMLHttpRequest();

  xmlhttp.onreadystatechange = function () {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("mensaje").innerHTML = this.responseText;
    }
  };

  xmlhttp.open("GET", "ajax_comprobar_nick.php?nickname=" + str, true);
  xmlhttp.send();
}
```

> El PHP recibe `$_GET['nickname']` (o `$_POST['nickname']` si se usa POST), consulta la base y devuelve un texto tipo `"Disponible"` o `"Ya existe"`.

---

17. AJAX con librerías (Prototype)

Existen frameworks que ya resuelven la instanciación y el uso de AJAX: **Prototype.js**, **jQuery**, **AngularJS**, **xajax** (PHP).

Ejemplo con `Ajax.Updater` de Prototype:

```
<script src="prototype.js" type="text/javascript"></script>
<script type="text/javascript">
function comprobar(nick) {
  var url = 'http://' + location.host + '/ajax/ajax_comprobar_nick.php';
  var pars = ("nickname=" + nick);

  var myAjax = new Ajax.Updater('comprobar_mensaje', url, {
    method: 'get',
    parameters: pars
  });
}
</script>

<input name="nickname" onkeyup="comprobar(this.value)">
<span id="comprobar_mensaje"></span>
```

Objeto Prototype	Función
`Ajax.Request(url, opciones)`	Realiza la petición y entrega la respuesta a un callback.
`Ajax.Updater(id, url, opciones)`	Realiza la petición y vuelca la respuesta en el elemento con ese `id`.
`parameters`	Datos a enviar (`"clave=valor"` o un objeto).
`method`	`'get'` o `'post'`.
`onSuccess` / `onFailure` / `onComplete`	Callbacks de resultado.

> Usar una librería **no obliga** a abandonar AJAX "a mano": se puede combinar.

---

18. Fetch API (método moderno)

Actualmente se usa un método basado en **promesas**. `fetch()` es más simple que `XMLHttpRequest`.

- `XMLHttpRequest` es la API heredada (aún útil para control estricto de la carga o muchas conexiones sincrónicas).
- `fetch()` es la API moderna para obtener datos de un servidor HTTP.

Con `.then()`

```
fetch(file)
  .then(function (response) {
    return response.text();
  })
  .then(function (data) {
    myDisplayer(data);
  });
```

Con funciones flecha

```
fetch(file)
  .then(response => response.text())
  .then(data => myDisplayer(data));
```

Con `async` / `await`

```
async function loadText(file) {
  const response = await fetch(file);
  myDisplayer(await response.text());
}

loadText("fetch.txt");
```

Objeto `Response`

`fetch()` devuelve una **promesa** que se resuelve en un objeto `Response`.

Propiedad / método	Para qué
`response.ok`	`true` si el estado HTTP está entre 200 y 299.
`response.status`	Código HTTP.
`response.statusText`	Texto del estado.
`response.text()`	Devuelve el cuerpo como texto (promesa).

Propiedad / método	Para qué
`response.json()`	Parsea el cuerpo como JSON (promesa).
`response.blob()`	Devuelve datos binarios.
`response.headers`	Cabeceras de la respuesta.

JSON

```
async function loadCustomer() {
  const response = await fetch("customer.json");
  const customer = await response.json();
  myDisplayer(customer.name);
}

loadCustomer();
```

Manejo de errores

```
async function cargar(url) {
  try {
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("HTTP " + response.status);
    }

    const datos = await response.json();
    console.log(datos);
  } catch (error) {
    console.error("Falló la petición:", error);
  }
}
```

> `fetch()` **no rechaza** la promesa ante un 404 o un 500: solo falla ante errores de red. Por eso hay que revisar `response.ok` a mano.

Enviar datos con Fetch

```
// JSON
await fetch("/api/usuarios", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ nombre: "Ana", edad: 25 })
});

// Formulario / archivos
const fd = new FormData(document.getElementById("form"));
await fetch("/subir.php", { method: "POST", body: fd });
```

	`XMLHttpRequest`	`fetch()`
Estilo	Callbacks y eventos	Promesas / `async-await`
Verbosidad	Alta	Baja
Progreso de descarga	`onprogress`	No directo (requiere streams)
Rechaza en 404/500	No	No (hay que mirar `ok`)
Cancelación	`abort()`	`AbortController`
Recomendación actual	Casos heredados	**Preferido**

19. Same Origin Policy y CORS

Same Origin Policy (Política del Mismo Origen): medida de seguridad que impide que un script de un origen acceda a recursos de otro origen distinto.

Dos URLs tienen el **mismo origen** si coinciden los tres componentes:

Componente	Ejemplo
Protocolo	`http` vs. `https`

Componente	Ejemplo
Host / dominio	`localhost` vs. `midominio.com`
Puerto	`:80` vs. `:8080`

```
http://localhost/app/index.html
http://localhost/app/datos.php      ✅ mismo origen
http://localhost:8080/datos.php     ❌ distinto puerto
https://localhost/datos.php         ❌ distinto protocolo
http://otro.com/datos.php           ❌ distinto dominio
```

- Por seguridad, los navegadores modernos **no permiten** el acceso entre dominios.
- La página y el archivo que se intenta cargar deben estar en el **mismo servidor**.
- Solución: **CORS** (*Cross-Origin Resource Sharing*), que el servidor habilita con cabeceras.

```
// En el servidor PHP
header("Access-Control-Allow-Origin: https://midominio.com");
header("Access-Control-Allow-Methods: GET, POST, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");
```

> La alternativa histórica eran los *scripts firmados*, pero la mayoría no están firmados y las versiones viejas de Internet Explorer no los soportan.

20. Códigos de estado HTTP

Rango	Categoría	Ejemplos
`1xx`	Informativo	`100 Continue`
`2xx`	Éxito	`200 OK`, `201 Created`
`3xx`	Redirección	`301`, `302`, `304`
`4xx`	Error del cliente	`400`, `401`, `403`, `404`
`5xx`	Error del servidor	`500`, `503`

En AJAX, `status == 200` es la verificación básica de éxito. Para uploads suele aceptarse también `201`.

21. Ventajas, desventajas y consideraciones

Ventajas

- Mejora la **experiencia de usuario**: sin recargas ni pantallas en blanco.
- **Respuestas más rápidas** para acciones simples.
- Se transfiere **solo lo necesario**, no páginas completas.
- Se puede **delegar el renderizado al cliente** y aliviar al servidor.
- Aplicaciones más **interactivas** tipo RIA (autocompletado, validaciones en vivo, carritos).

Desventajas y consideraciones

Problema	Detalle	Posible solución
Historial del navegador	Las páginas creadas dinámicamente no se registran; "Volver" no restaura el estado anterior.	`History API` (`pushState`), *iframes* invisibles.
Motores de búsqueda	No analizan JavaScript; el contenido dinámico no se indexa.	Renderizado en servidor, contenido accesible por URL.
Same Origin Policy	Bloquea peticiones entre dominios distintos.	CORS.
Sobrecarga del servidor	Muchas llamadas AJAX pueden saturarlo.	Cachear, agrupar peticiones, *debounce*.
Dispositivos	Puede no funcionar en teléfonos viejos, PDA u otros aparatos.	Diseño progresivo.
Accesibilidad	No es compatible con todos los lectores de pantalla.	Alternativas sin JS, ARIA.
Dependencia de JS	Si JavaScript falla o está deshabilitado, la app no funciona.	Contenido base en HTML.

22. Errores típicos y checklist

Error	Causa	Solución
`readyState == 4` pero no pasa nada	Falta verificar `status == 200`.	Agregar la condición de estado.
El callback nunca se ejecuta	Se asignó el handler **después** de `send()`.	Asignar `onreadystatechange` antes de `send()`.
El POST llega vacío al servidor	Falta `Content-Type` o los datos no están serializados.	`setRequestHeader("Content-Type", "application/x-www-form-urlencoded")` y `send("a=1&b=2")`.
El navegador bloquea la petición	Distinto origen (protocolo, host o puerto).	Servir todo desde el mismo origen o habilitar CORS.
`responseText` vacío	Se quiso leer en `readyState == 3`.	Leer en `readyState == 4`.
Funciona en local pero no en el servidor	Se abrió con `file://`.	Usar `http://localhost/...`.
Se muestran etiquetas HTML como texto	Se usó `textContent` o el servidor escapó el HTML.	Usar `innerHTML` si corresponde.
El acento se ve mal	Codificación distinta entre cliente y servidor.	UTF-8 en ambos lados.
La página "se cuelga"	Se usó `async = false`.	Usar `true`.
`fetch` no detecta el 404	`fetch` no rechaza en errores HTTP.	Verificar `response.ok`.
Cambios no visibles	Se modificó el elemento equivocado o no existe aún.	Revisar `id` y ejecutar después del `DOMContentLoaded`.

Checklist antes de entregar

- [ ] `XMLHttpRequest` creado antes de usarlo (con verificación de navegador).
- [ ] `onreadystatechange` (u `onload`) asignado **antes** de `send()`.
- [ ] Se verifica `readyState == 4` **y** `status == 200`.
- [ ] `open()` con el método correcto: GET para consultar, POST para modificar.
- [ ] POST con `Content-Type: application/x-www-form-urlencoded`.
- [ ] `async = true`.
- [ ] URL correcta y accesible desde el mismo origen.
- [ ] La respuesta se inserta en el contenedor correcto (`innerHTML` / `textContent`).
- [ ] Manejo de errores (`onerror`, `try/catch` o `.catch()`).
- [ ] Funciona con y sin datos; probado el caso de error.
- [ ] Codificación UTF-8 en cliente, servidor y base de datos.

---

23. Plantilla completa

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>AJAX — Plantilla</title>
</head>
<body>

  <button type="button" onclick="cargar('datos.txt')">Cargar</button>
  <div id="salida"></div>

  <script>
  function cargar(url) {
    const salida = document.getElementById("salida");

    // 1. Crear el objeto
    const xhttp = new XMLHttpRequest();

    // 2. Preparar la respuesta
    xhttp.onreadystatechange = function () {
      if (this.readyState !== 4) return;

      if (this.status === 200) {
        salida.innerHTML = this.responseText;
```

```
      } else {
        salida.textContent = "Error HTTP " + this.status;
      }
    };

    xhttp.onerror = function () {
      salida.textContent = "Error de red";
    };

    // 3. Realizar la petición
    xhttp.open("GET", url, true);
    xhttp.send();
  }

  // Variante POST
  function enviar(url, datos) {
    const xhttp = new XMLHttpRequest();

    xhttp.onreadystatechange = function () {
      if (this.readyState === 4 && this.status === 200) {
        document.getElementById("salida").innerHTML = this.responseText;
      }
    };

    xhttp.open("POST", url, true);
    xhttp.setRequestHeader("Content-Type",
                           "application/x-www-form-urlencoded");
    xhttp.send(datos);   // "clave1=valor1&clave2=valor2"
  }

  // Variante moderna con Fetch
  async function cargarModerno(url) {
    try {
      const response = await fetch(url);
      if (!response.ok) throw new Error("HTTP " + response.status);
      document.getElementById("salida").textContent = await response.text();
    } catch (error) {
      console.error(error);
      document.getElementById("salida").textContent = "Error al cargar";
    }
  }
  </script>

</body>
</html>
```

Fin del cheatsheet.
Si tenés que priorizar para el parcial: qué es AJAX y qué tecnologías lo componen, diferencias con el modelo tradicional, los cuatro bloques de una aplicación AJAX, `open()` / `send()`, `readyState` (0–4) y `status`, `onreadystatechange` vs. `onload`, GET vs. POST, Same Origin Policy/CORS y `fetch()`.